

CS3.301: Operating Systems and Networks

Lecture 6: CPU Scheduling (Multi-Level Feedback Queue)

INSTRUCTOR: DR. DEEPAK GANGADHARAN

Today: The Multi-Level Feedback Queue

- **The problem:** schedule without knowing a job's length in advance
- **Attempt #1** — changing a job's priority based on observed behavior
- **Attempt #2** — the periodic priority boost (fixing starvation)
- **Attempt #3** — better accounting (fixing gaming of the scheduler)
- **Tuning MLFQ**, and how real systems (Solaris, BSD, Windows NT) use it

How Do We Schedule Without Perfect Knowledge?

“How can we design a scheduler that both minimizes response time for interactive jobs while also minimizing turnaround time — without a-priori knowledge of job length?”

- **Goal 1 — Turnaround time:** best achieved by running shortest jobs first (SJF / STCF) — but the OS doesn't know job length up front.
- **Goal 2 — Response time:** Round Robin is responsive, but terrible for turnaround time.
- **The question:** can a scheduler learn a job's characteristics as the system runs, and adapt?

Tip: Learn From History

- MLFQ is a classic example of a system that learns from the past to predict the future.
- **Familiar pattern:** the same idea drives hardware branch predictors and caching algorithms.
- **When it works:** jobs have phases of behavior that are reasonably predictable.
- **The catch:** history-based heuristics can be wrong — and can drive worse decisions than having no knowledge at all.

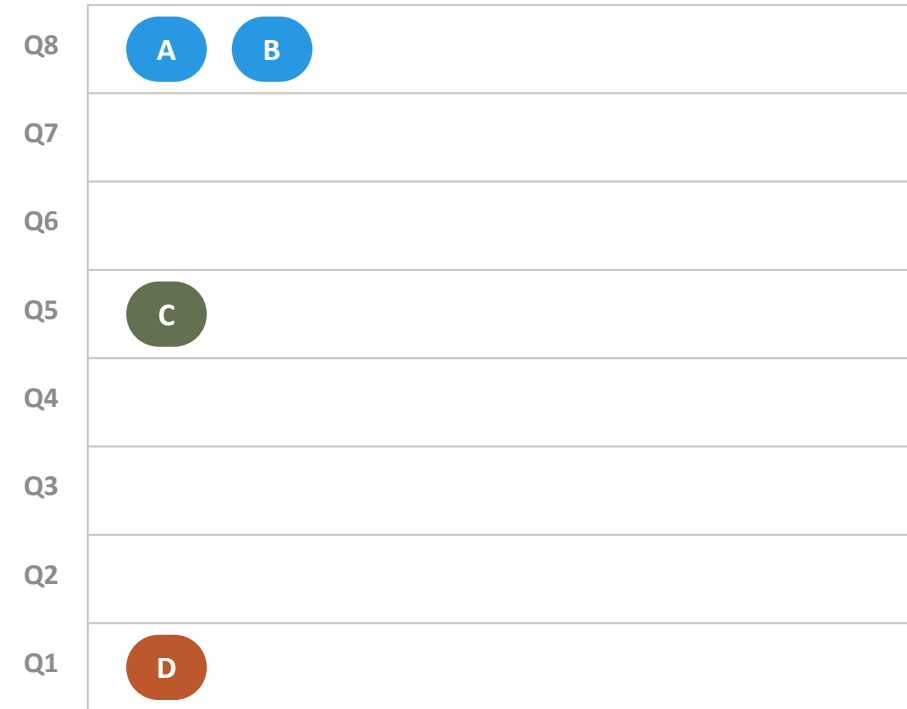
Rule 1 & 2: Priority Decides Who Runs

Rule 1: If $\text{Priority}(A) > \text{Priority}(B)$, A runs (B doesn't).

Rule 2: If $\text{Priority}(A) = \text{Priority}(B)$, A & B run in Round-Robin.

- MLFQ has several distinct queues, each at a different priority level.
- A ready job sits on exactly one queue at any given time.
- More than one job may share a queue — they round-robin among themselves.

Figure 8.1 — A Snapshot of MLFQ Queues
High Priority



Low Priority

The Key Idea: Watch and Adapt

- MLFQ does not assign a fixed priority — it varies priority based on observed behavior.
- **Relinquishes the CPU often (e.g. waiting on the keyboard)?** priority stays high — looks interactive.
- **Uses the CPU intensively for long stretches?** priority is lowered — looks batch-like.
- In this way MLFQ learns about a process as it runs, and uses that history to predict its future behavior.

The Allotment

The allotment is the amount of time a job may spend at a given priority level before the scheduler demotes it.

- For simplicity, we'll start by assuming the allotment equals a single time slice.
- The allotment is what makes the priority-adjustment rules (next slide) precise and enforceable.

How To Change Priority

Rule 3: When a job enters the system, it is placed at the highest priority (the topmost queue).

Rule 4a: If a job uses up its allotment while running, its priority is reduced (moves down one queue).

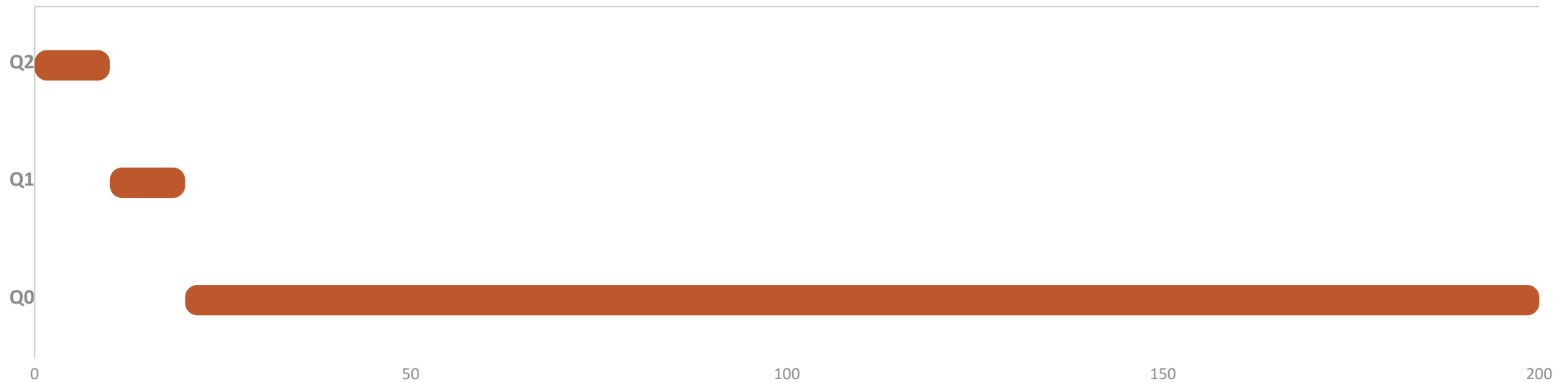
Rule 4b: If a job gives up the CPU (e.g. for I/O) before the allotment is up, it stays at the same priority level.

Workload assumption: a mix of short interactive jobs and long CPU-bound jobs, with a 10 ms time slice used in the examples that follow.

EXAMPLE 1

A Single Long-Running Job

- Three-queue system, 10 ms time slice (= allotment) at every level.
- The job enters at the highest priority, Q2.
- After one time slice (10 ms) it is demoted to Q1, then after another to Q0 — where it stays.



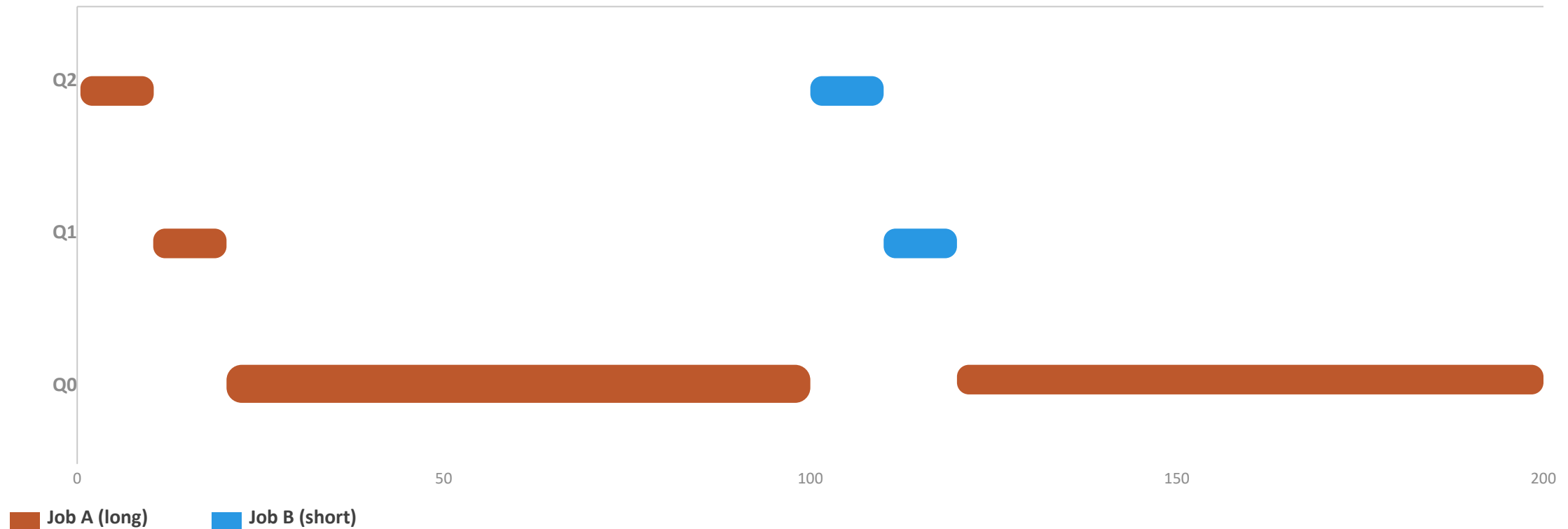
The job steps down one queue per time slice until it settles at Q0.

Along Came a Short Job

- **Job A** — long-running, CPU-intensive — has been running for a while at the lowest queue.
- **Job B** — short, interactive (20 ms total) — arrives at $T = 100$ and enters the highest queue.
- Because B is short, it finishes in just two time slices — before it ever drops far down the queues.

MLFQ doesn't know in advance whether a job is short or long — so it assumes “short” first. A truly short job finishes quickly at high priority; a long job reveals itself by sinking through the queues. This is how MLFQ approximates SJF.

Approximating SJF Without Apriori Knowledge



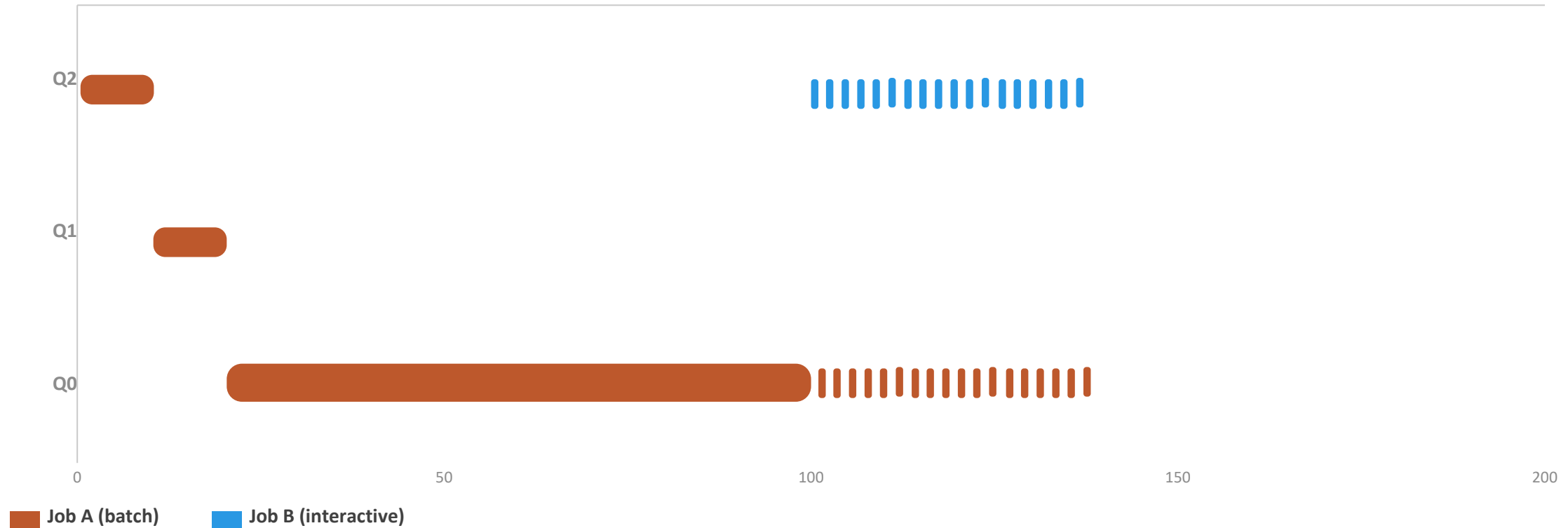
B arrives at $T=100$, runs to completion in two slices at high priority, then A resumes at Q0.

What About I/O?

Recall Rule 4b: give up the CPU before the allotment is used — and priority is preserved.

- An interactive job that does a lot of I/O (keyboard, mouse) relinquishes the CPU frequently.
- Rule 4b means we don't penalize it for that — it simply stays at the same priority level.
- Net effect: genuinely interactive jobs remain at the top and stay responsive.

An Interactive Job That Keeps Yielding



B needs only ~1 ms before I/O each time, so it stays pinned at the highest queue while A runs in the background.

Problems With Our Current MLFQ

1

Starvation

Too many interactive jobs can consume all CPU time — long-running jobs never run.

2

Gaming the Scheduler

A sneaky job can issue I/O just before its allotment ends to keep its high priority.

3

Behavior Changes

A job that turns from CPU-bound to interactive is never recognized as such.

Starvation

- If there are “too many” interactive jobs in the system, they combine to consume all CPU time.
- Long-running (CPU-bound) jobs may then never receive any CPU — they starve.
- We’d like a guarantee of at least some progress for these jobs, even in the worst case.

Gaming the Scheduler

- “Gaming” = doing something sneaky to grab more than your fair share of a resource.
- **The exploit:** issue an I/O operation just before the allotment is used up (e.g. at 99% of it).
- Rule 4b then keeps the job at the same high priority — letting it near-monopolize the CPU.

Tip — Scheduling Must Be Secure: in shared datacenters, scheduling policy is a security boundary; a careless design lets one user harm others.

A Job's Behavior Can Change Over Time

- A program may start CPU-bound and later shift into an interactive phase (or vice-versa).
- With our rules so far, a job that has sunk to a low-priority queue stays there — permanently.
- It is never treated like the other interactive jobs, even after its behavior has genuinely changed.

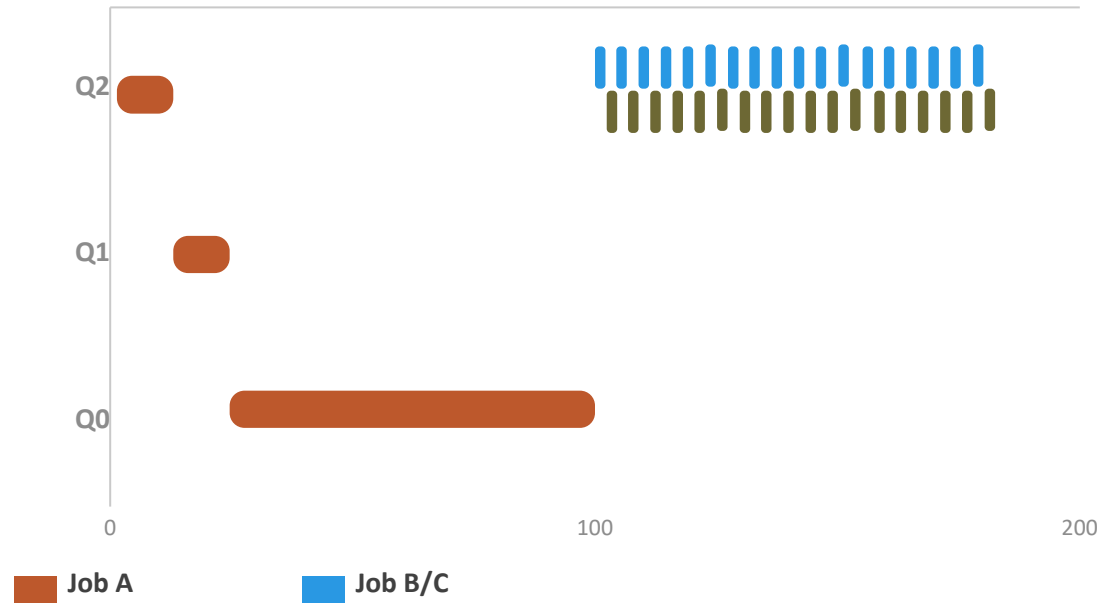
The Priority Boost

Rule 5: After some time period S , move all the jobs in the system to the topmost queue.

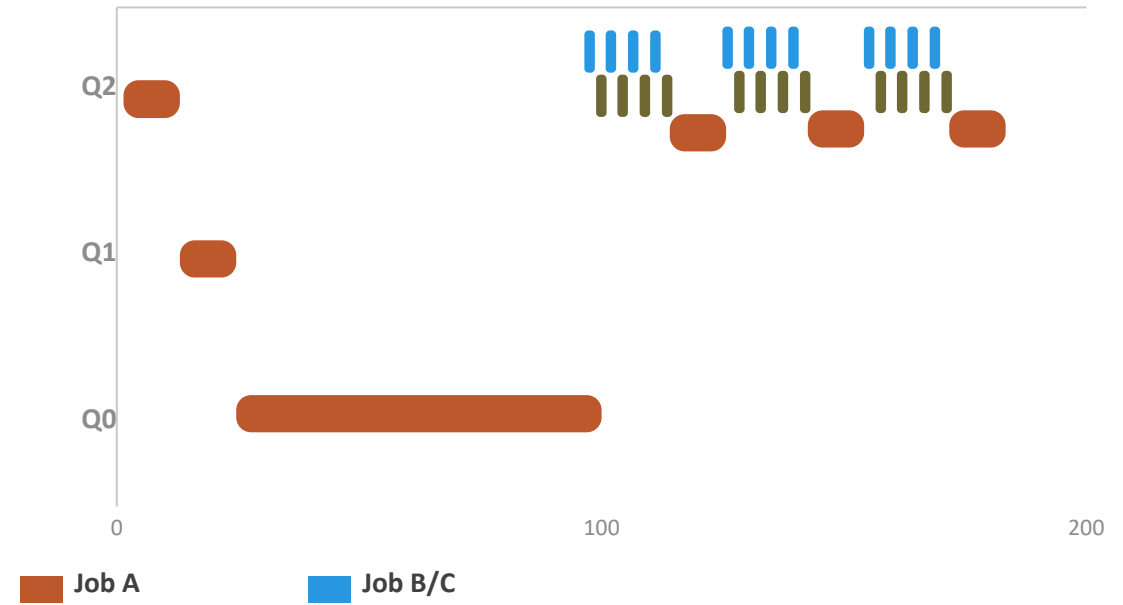
- **Fixes starvation:** sitting in the top queue guarantees a job will round-robin with other high-priority jobs and eventually get service.
- **Fixes behavior changes:** a job that has become interactive is treated properly again right after a boost.

Before and After a Periodic Boost

Without boost — A starves



With boost every 100 ms — A makes progress



a boost every 100 ms (deliberately small here, for illustration) guarantees the long job A gets scheduled periodically.

Voodoo Constants (Ousterhout's Law)

- What should S (the boost period) be set to?
- **Too high:** long-running jobs can still starve between boosts.
- **Too low:** interactive jobs lose their fair share of the CPU.
- John Ousterhout called such values “voo-doo constants” — they seem to need black magic to set correctly.
- Often left to a config file a sysadmin rarely touches — or, increasingly, tuned automatically via ML.

Better Accounting

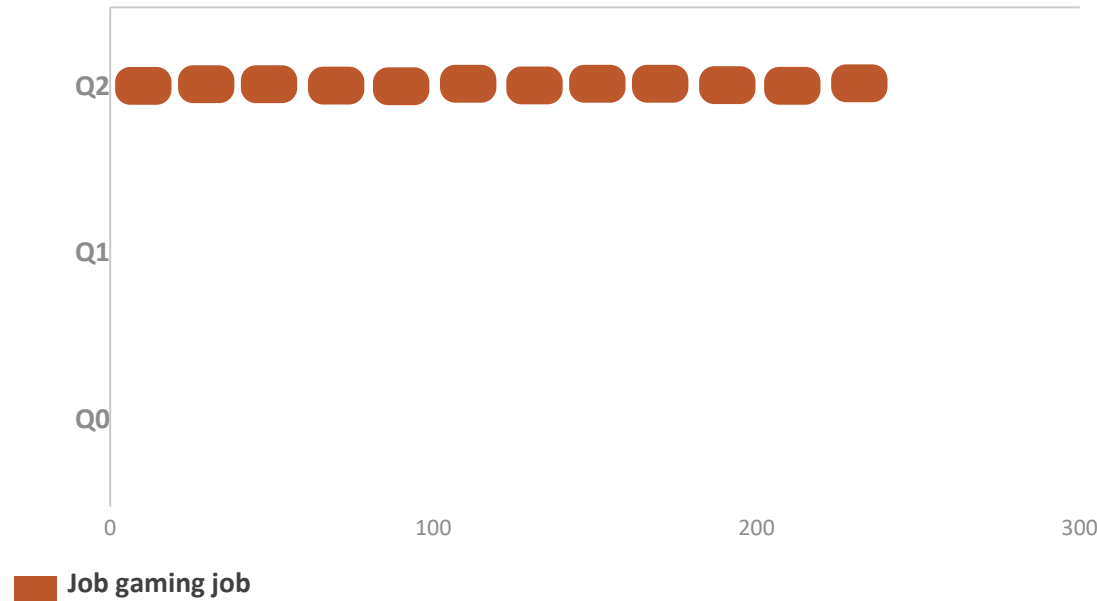
- The real culprit is Rules 4a/4b: they let a job “refresh” its allotment just by giving up the CPU before it's used.
- Fix: track total CPU time consumed at a level across all bursts — not just the most recent one.

New Rule 4: Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced.

This single rule replaces the old Rules 4a and 4b.

Gaming Tolerance: Before and After

Old Rules 4a/4b — gaming works



New Rule 4 — accounted, cannot game

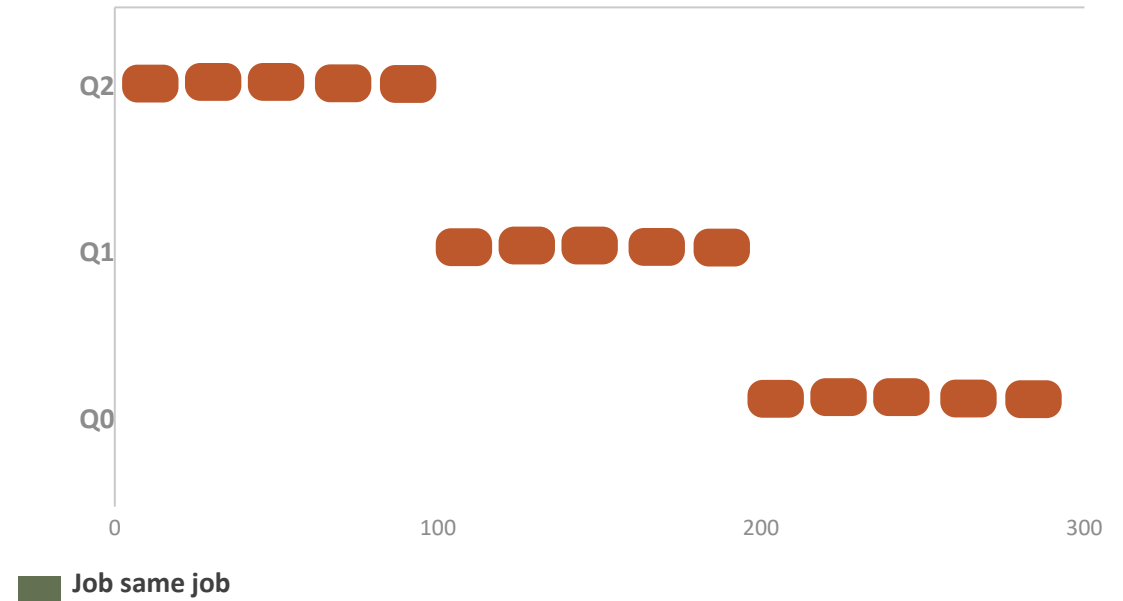


Figure 8.5 — with accurate accounting, a job is demoted on schedule regardless of how it slices its I/O.

Tuning MLFQ: Queues, Slices, Allotments

- How many queues should there be? How long is the time slice at each level? The allotment? How often to boost?
- **High-priority queues:** short time slices (≤ 10 ms) — they hold interactive jobs, so fast alternation feels responsive.
- **Low-priority queues:** longer time slices (100s of ms) — they hold CPU-bound jobs, so context-switch overhead matters less.
- There is no universal answer — tuning comes from experience with real workloads.

Longer Quanta at Lower Priority

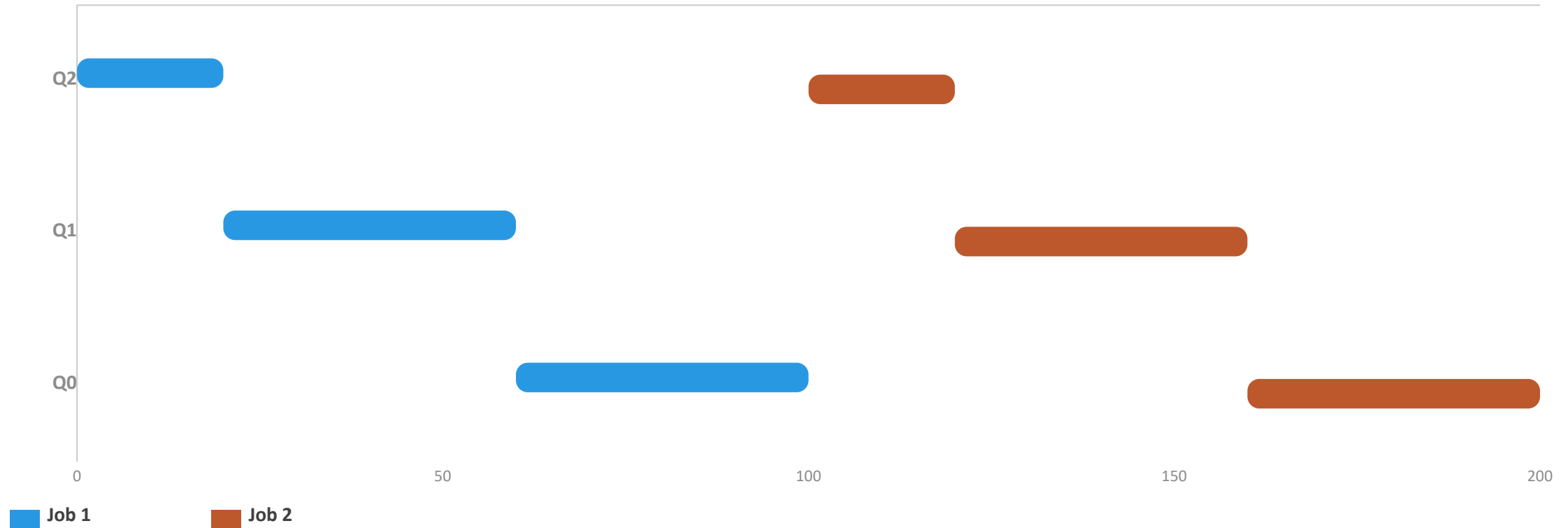


Figure 8.6 — 10 ms slices at Q2, 20 ms at Q1, 40 ms at Q0: each job runs 20 ms high, 40 ms mid, 40 ms low.

Real-World MLFQ Schedulers

- **Solaris Time-Sharing (TS) class:** 60 queues by default, tunable tables for time-slice length and priority changes, boosted roughly every second.
- **FreeBSD (4.3):** uses a formula based on decayed CPU usage rather than an explicit table — same spirit, different mechanism.
- **Also used by:** BSD UNIX derivatives, and Windows NT and its successors.
- **User advice:** the nice command lets a user raise or lower a job's priority, giving the OS a hint it need not obey.

Summary: The Five Rules of MLFQ

Rule 1: If $\text{Priority}(A) > \text{Priority}(B)$, A runs.

Rule 2: If $\text{Priority}(A) = \text{Priority}(B)$, A & B run in Round-Robin.

Rule 3: A new job enters at the highest priority (topmost queue).

Rule 4: Once a job uses its allotment at a level (however it was split up), it is demoted.

Rule 5: After time period S, boost every job back to the topmost queue.

MLFQ needs no a-priori knowledge of a job, yet delivers SJF/STCF-like turnaround for short interactive jobs and fair, starvation-free progress for long CPU-bound jobs.

Thank You!!